PARAMETER-EFFICIENT FINE-TUNING · WORKED EXAMPLE

Prompt Tuning & Prefix Tuning

Both freeze the entire pretrained model and learn only a handful of new vectors. The difference is *where those vectors get inserted* — and that one design choice changes how much they can do.

Full fine-tuning updates every weight in a language model — hundreds of millions to hundreds of billions of numbers — to adapt it to one task. Parameter-Efficient Fine-Tuning (PEFT) asks: what's the smallest number of *new* parameters we can add, while leaving the original weights untouched, and still steer the model? Several answers exist:

adapters	small bottleneck MLPs inserted between existing layers
LoRA	a low-rank update `ΔW = BA` added alongside frozen weight matrices
prompt tuning	learned vectors prepended to the input embeddings, nowhere else
prefix tuning	learned vectors injected into the attention of *every* layer

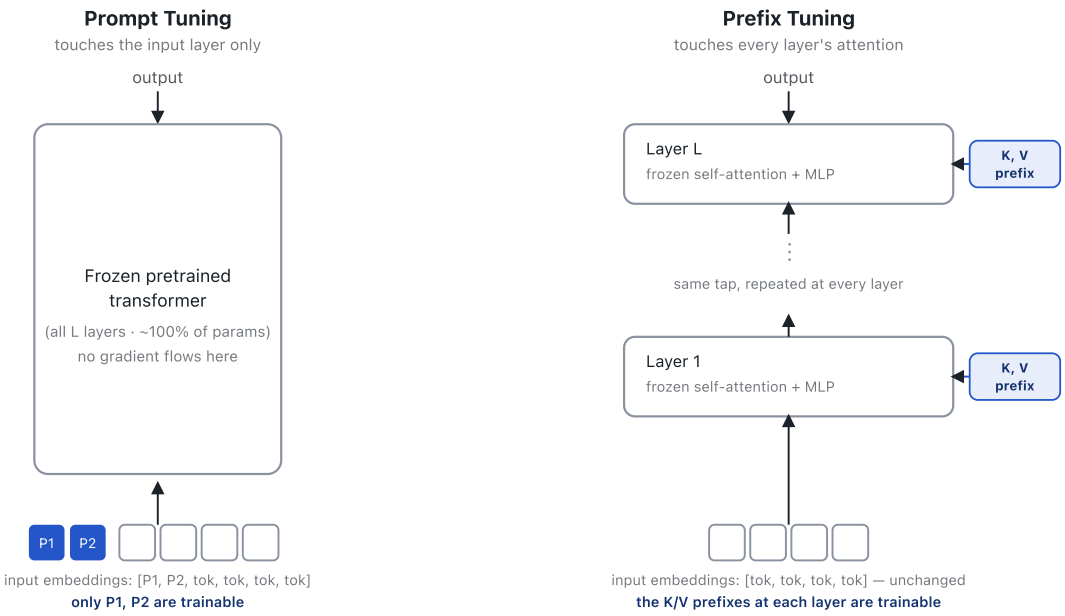
This page works through the last two in detail, with real model dimensions and a fully worked numeric example for each.

SHARED MECHANISM

Freeze everything, train a whisper

In both methods the base transformer's weights — every attention matrix, every feed-forward layer, the embedding table — are set to `requires_grad = False`. Nothing about them changes during training. The only parameters that receive a gradient are a small set of freshly initialized vectors. At inference, the frozen model runs exactly as it always has, except it now also sees these extra vectors sitting in its input.

trainable — receives gradients ▢ frozen — pretrained, untouched



What changes between the two. Prompt tuning adds trainable vectors once, at the bottom of the stack, and lets the ordinary frozen forward pass carry their influence upward. Prefix tuning gives up that simplicity for direct access: a separate small set of trainable key/value vectors is injected into the attention computation at *every* layer, so the signal doesn't have to survive being propagated through many frozen layers to reach the top.

METHOD ONE

Prompt tuning

Prompt tuning (Lester et al., 2021) prepends `p` learned embedding vectors — a *soft prompt* — directly to the input token embeddings. These vectors don't correspond to any word in the vocabulary; they're free-floating points in embedding space, initialized randomly (or from sampled vocabulary embeddings) and then optimized by gradient descent, exactly like any other embedding would be if it weren't frozen. Everything downstream — every attention layer, every feed-forward block — processes them exactly as it processes real token embeddings, but none of those downstream weights change.

Worked example: sentiment classification on GPT-2

Take GPT-2 small: `d_model = 768`, 12 layers, ≈124M parameters. Choose a soft

prompt of length `p = 20`. Trainable parameter count is just:

```
trainable = p × d_model
trainable = 20 × 768 = 15,360 # a 20×768 matrix, nothing else
```

15,360	124,000,000	≈ 0.012%
trainable parameters	frozen parameters	of the model is trained

Now trace an actual sentence through it. Say the task is sentiment classification, the input is `"the movie was great"`, and — to keep the numbers readable — the embedding dimension is shrunk to a toy size of 6 (GPT-2's real embeddings are 768-wide; the mechanism is identical, just wider). Two soft-prompt vectors, `P1` and `P2`, are prepended ahead of the four real token embeddings:

POSITION	TOKEN	EMBEDDING (TOY 6-DIM), BEFORE → AFTER TRAINING	STATUS
1	P1 *(soft prompt)*	`[ 0.12, −0.08, 0.31, 0.02, −0.19, 0.07 ]` → **`[ 0.41, −0.33, 0.58, −0.12, −0.47, 0.29 ]`**	TRAINABLE
2	P2 *(soft prompt)*	`[−0.05, 0.22, −0.14, 0.36, 0.01, −0.09]` → **`[−0.28, 0.51, −0.39, 0.62, 0.18, −0.31]`**	TRAINABLE
3	"the"	`[ 0.11, 0.04, −0.02, 0.09, −0.01, 0.03 ]` → `[ 0.11, 0.04, −0.02, 0.09, −0.01, 0.03 ]` *(identical)*	FROZEN
4	"movie"	`[−0.07, 0.15, 0.22, −0.03, 0.08, −0.11]` → `[−0.07, 0.15, 0.22, −0.03, 0.08, −0.11]` *(identical)*	FROZEN
5	"was"	`[ 0.02, −0.01, 0.05, 0.01, −0.02, 0.00 ]` → `[ 0.02, −0.01, 0.05, 0.01, −0.02, 0.00 ]` *(identical)*	FROZEN
6	"great"	`[ 0.33, 0.18, −0.05, 0.21, 0.14, −0.02 ]` → `[ 0.33, 0.18, −0.05, 0.21, 0.14, −0.02 ]` *(identical)*	FROZEN

Illustrative values, not weights from an actual trained checkpoint — chosen only to show which numbers move.

The sequence `[P1, P2, the, movie, was, great]` is fed into the frozen 12-layer transformer exactly like any other input. During backpropagation, the loss (say, from a classification head predicting *positive/negative* on top of the final hidden state) sends gradients back through all 12 frozen layers — but every one of GPT-2's own weight matrices has `requires_grad = False`, so the optimizer skips them. Only rows 1 and 2, `P1` and `P2`, get updated. After enough steps, those two vectors settle into whatever shape makes the frozen model — without changing a single one of its own weights — reliably output "positive" for sentences like this one.

The trade-off: prompt tuning's soft prompt only ever touches the model at the very first layer. Everything it wants to say to layer 11 has to survive being processed by layers 1 through 10 first — which is why the original paper found it needs a genuinely large frozen model (billions of parameters) before it matches full fine-tuning quality. Below that scale it tends to underperform.

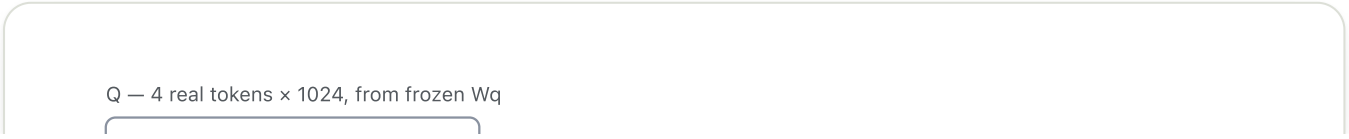
METHOD TWO

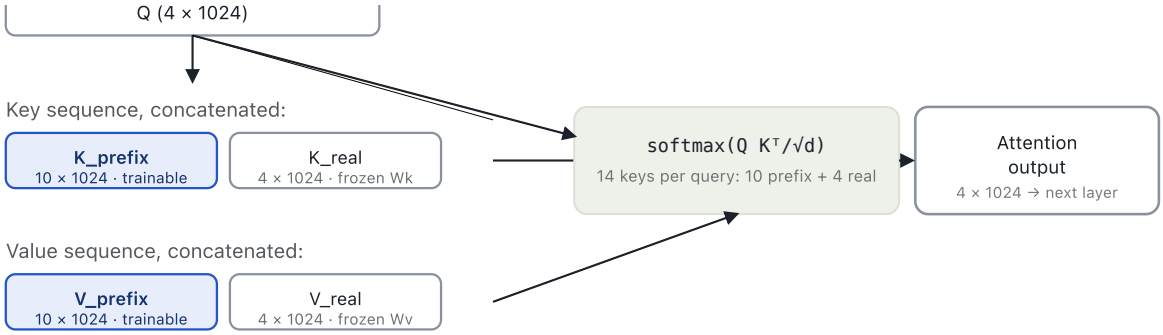
Prefix tuning

Prefix tuning (Li & Liang, 2021) keeps the same principle — freeze the model, train a small vector set — but changes where those vectors act. Instead of one insertion point at the input embeddings, every attention layer gets its own private pair of trainable key/value vectors, prepended directly to that layer's keys and values before the attention softmax runs. The real tokens' queries can now attend over those extra "memory slots" at every layer, not just the first.

The mechanism, inside one layer

For a layer processing `n = 4` real tokens with `d_model = 1024` (GPT-2 Medium's real width) and a chosen prefix length `p = 10`: the real keys and values, `K_real` and `V_real`, are computed the normal, frozen way — shape `(4, 1024)` each. Alongside them sits a trainable pair `K_prefix`, `V_prefix`, shape `(10, 1024)` each, unique to this layer. Attention runs over the concatenation of the two:





Only the blue boxes learn. `W_q`, `W_k`, `W_v`, and the real tokens' resulting `K_real`/`V_real` are entirely frozen — they're the same values GPT-2 always produces. Each real token's query can now also attend to the 10 prefix "memory" positions. Since `K_prefix` and `V_prefix` are the only thing that can change, they're also the only lever training has on this layer's output — and every layer gets its own independent lever.

Worked example: GPT-2 Medium

Scale that up to a full model — GPT-2 Medium, `L = 24` layers, `d_model = 1024`, ≈345M parameters — with prefix length `p = 10`. Each layer needs its own `K_prefix` and `V_prefix`, so the count is layers × prefix length × dimension × 2:

```
trainable = L × p × d_model × 2
trainable = 24 × 10 × 1024 × 2 = 491,520 # 24 independent (K,V)
pairs, one per layer
```

491,520	345,000,000	≈ 0.14%
trainable parameters	frozen parameters	of the model is trained

That's roughly **32×** more trainable parameters than the prompt-tuning example above — still a rounding error next to the frozen 345M — but because it's spent as small, direct taps into every layer instead of one tap at the bottom, it tends to reach good task performance at far smaller model scales than prompt tuning does. In the original paper, the vectors aren't optimized directly either: a small trainable MLP re-parameterizes a smaller trainable matrix into the full prefix during training, purely to stabilize optimization. Once training finishes, the MLP is discarded and only its output — the plain prefix vectors — is kept for inference, so deployment cost matches the numbers above exactly.

WHERE THEY SIT

In the PEFT family

METHOD	WHAT'S TRAINABLE	FORMULA	TYPICAL SHARE	INFERENCE COST
Full fine-tuning	every weight in the model	`100%`	100%	none, but a full copy per task
Prompt tuning	`p` soft-prompt embeddings, input layer only	`p · d`	~0.001–0.05%	a few extra tokens per pass
Prefix tuning	`K, V` prefix pair at every layer's attention	`L · p · d · 2`	~0.05–3%	extra keys/values per layer
LoRA	low-rank update `ΔW = BA` on chosen weight matrices	`2 · r · d (per matrix)`	~0.1–1%	none — `ΔW` merges into `W`
Adapters	small bottleneck MLP after each sublayer	`2 · d · r (per adapter)`	~1–4%	small added latency, can't merge

> **RULE OF THUMB**
>
> Prompt tuning is the cheapest and simplest option, but its one-shot insertion at the input needs a very large frozen model to work well. Prefix tuning spends a bit more capacity — still far under 1% of the model — in exchange for a direct line into every layer, which makes it more reliable at moderate model sizes and on generation tasks. Neither touches inference-time weights, so a single frozen base model can serve many tasks at once by swapping in a different small vector set per request.

Prompt tuning: Lester, Al-Rfou & Constant, *"The Power of Scale for Parameter-Efficient Prompt Tuning,"* 2021. Prefix tuning: Li & Liang, *"Prefix-Tuning: Optimizing Continuous Prompts for Generation,"* 2021. Model dimensions (GPT-2 small/medium) are the publicly documented architecture sizes, used here to make the

dimensions (GPT-2 small/medium) are the publicly documented architecture sizes, used here to make the parameter-count arithmetic concrete.